CAHIER DES CHARGES — BACKEND

Projet JobLink | Spécification détaillée de l'API et des données

Projet	JobLink — Plateforme de recrutement
Périmètre	API, règles métier, sécurité et base de données
Technologies	Node.js · Express.js · JavaScript · PostgreSQL
API	REST JSON · documentation OpenAPI/Swagger
Version	1.0 — Novembre 2026

Sommaire

• 1. Présentation et périmètre

• 2. Acteurs, rôles et permissions

• 3. Stack et architecture technique

• 4. Organisation recommandée des fichiers

• 5. Modèle de données PostgreSQL

• 6. Authentification et gestion des comptes

• 7. Gestion des entreprises et des offres

• 8. Gestion des candidatures et entretiens

• 9. CV, fichiers et favoris

• 10. Notifications et communication

• 11. Contrat API et gestion des erreurs

• 12. Sécurité et protection des données

• 13. Tests, documentation et observabilité

• 14. Déploiement et configuration

• 15. Critères d'acceptation et ordre de réalisation

1. Présentation et périmètre

Ce document définit exclusivement le **backend** de JobLink. Le backend expose une API REST pour l'application React, applique les règles métier, authentifie les utilisateurs, contrôle les permissions et gère les données dans PostgreSQL.

Technologies imposées : **Node.js, Express.js, JavaScript uniquement et PostgreSQL**. Aucun TypeScript. Les exemples d'architecture et de routes ci-dessous constituent la base de réalisation et devront être ajustés après validation des règles métier.

Objectifs

- Fournir des endpoints REST cohérents et documentés.
- Garantir confidentialité, intégrité et contrôle d'accès aux données.
- Gérer les workflows de publication des offres et de candidature.
- Assurer validation, journalisation, tests et maintenabilité.

2. Acteurs, rôles et permissions

Rôle	Responsabilités principales
Visiteur	Consulter les offres publiques, créer un compte et se connecter.
Candidat	Gérer son profil/CV, sauvegarder des offres, postuler et suivre ses candidatures.
Recruteur	Gérer les offres et candidatures des entreprises auxquelles il est autorisé.
Administrateur	Administrer les comptes, modérer les entreprises/offres et consulter les indicateurs autorisés.

Mettre en place un contrôle d'accès basé sur les rôles (RBAC) et, lorsque nécessaire, sur la propriété de la ressource. Un recruteur ne doit pas pouvoir gérer les offres d'une entreprise dont il n'est pas membre autorisé.

3. Stack et architecture technique

- Node.js version LTS et Express.js.
- JavaScript CommonJS ou ES Modules : choisir une convention et la conserver partout.
- PostgreSQL avec un client/ORM unique (par exemple node-postgres ou Prisma en JavaScript).
- JWT ou session sécurisée selon l'architecture retenue ; bcrypt pour le hachage des mots de passe.
- Multer pour les fichiers, avec stockage local en développement et stockage objet sécurisé en production si nécessaire.
- Jest et Supertest pour les tests ; Swagger/OpenAPI pour la documentation.
- Variables d'environnement via dotenv ; logs structurés.

Architecture en couches

- Routes : déclaration des endpoints et middlewares.
- Controllers : lecture de req, appel du service et construction de la réponse HTTP.
- Services : règles métier et orchestration.
- Models/Repositories : accès aux données PostgreSQL.

• Middlewares : authentification, autorisation, validation, gestion d'erreurs et upload.

• Config : base de données, JWT, stockage et paramètres d'application.

4. Organisation recommandée des fichiers

```
src/
config/ (database.js, env.js, swagger.js)
routes/ (authRoutes.js, jobRoutes.js, applicationRoutes.js)
controllers/
services/
models/ ou repositories/
middlewares/ (auth, authorize, validate, errorHandler, upload)
validators/
utils/ (tokens, mail, pagination, logger)
jobs/ (tâches planifiées si nécessaires)
app.js
server.js
database/
migrations/
seeds/
tests/
.env.example
README.md
```

Séparer app.js (configuration Express et middlewares) de server.js (démarrage du serveur et écoute du port) pour faciliter les tests.

5. Modèle de données PostgreSQL

Table	Champs principaux / rôle
users	id, first_name, last_name, email UNIQUE, password_hash, role, status, created_at, updated_at.
candidate_profiles	user_id FK, phone, location, biography, photo_url.
companies	id, name, description, website, logo_url, address, status.
company_members	company_id FK, user_id FK, company_role, status.
skills	id, name UNIQUE.
candidate_skills	candidate_id FK, skill_id FK, level.
experiences	id, candidate_id FK, title, employer, dates, description.
educations	id, candidate_id FK, institution, degree, dates.
resumes	id, candidate_id FK, file_url, file_name, mime_type, size, is_primary.
job_offers	id, company_id FK, recruiter_id FK, title, description, contract_type, location, salary_min/max, status, published_at, deadline.
job_skills	job_id FK, skill_id FK, required_level.
applications	id, job_id FK, candidate_id FK, resume_id FK, cover_letter, status, applied_at.
application_history	id, application_id FK, old_status, new_status, changed_by, note, created_at.
interviews	id, application_id FK, scheduled_at, format, location_or_link, status, notes.

Table	Champs principaux / rôle
notifications	id, user_id FK, type, title, message, read_at, created_at.
saved_jobs	user_id FK, job_id FK, created_at; clé unique composée.
activity_logs	id, actor_id, action, entity_type, entity_id, metadata, created_at.

Contraintes : clés primaires, clés étrangères, index sur email/statut/date/relations de recherche, unicité des emails et des favoris, règles ON DELETE explicites, timestamps et migrations versionnées. Les suppressions sensibles peuvent être logiques (status/archived_at) plutôt que physiques.

6. Authentification et gestion des comptes

Méthode et route	Règle attendue
POST /api/auth/register	Valider les champs, vérifier l'unicité email, hacher le mot de passe, créer le compte et le profil initial.
POST /api/auth/login	Vérifier identifiants, statut du compte et retourner session/token selon stratégie.
POST /api/auth/refresh	Renouveler l'accès si refresh token valide, non révoqué et non expiré.
POST /api/auth/logout	Révoquer le refresh token ou invalider la session.
GET /api/auth/me	Retourner l'utilisateur connecté et les informations de rôle autorisées.
POST /api/auth/forgot-password	Créer un jeton de réinitialisation à durée limitée et envoyer le lien si service mail configuré.
POST /api/auth/reset-password	Vérifier le jeton, modifier le hash du mot de passe et invalider les sessions selon politique.

- Ne jamais stocker les mots de passe en clair.
- Utiliser bcrypt avec un coût adapté et configurable.
- Normaliser les emails (trim + lowercase) et imposer une validation serveur.
- Limiter les tentatives de connexion et protéger les endpoints sensibles contre les abus.
- Ne pas révéler si un email existe lors de la demande de réinitialisation.

7. Gestion des entreprises et des offres

Endpoint	Fonction
GET /api/companies/:id	Consulter les informations publiques d'une entreprise.
POST /api/companies	Créer une entreprise pour un utilisateur autorisé.
PATCH /api/companies/:id	Modifier les informations si l'utilisateur est membre autorisé.
POST /api/companies/:id/members	Inviter/ajouter un membre selon permissions.
GET /api/jobs	Lister les offres publiques avec pagination, filtres et tri.
GET /api/jobs/:id	Afficher une offre publiée et ses informations publiques.
POST /api/jobs	Créer une offre pour une entreprise autorisée.

Endpoint	Fonction
PATCH /api/jobs/:id	Modifier une offre appartenant à l'entreprise autorisée.
PATCH /api/jobs/:id/publish	Publier une offre après vérification des champs requis.
PATCH /api/jobs/:id/close	Clôturer une offre selon les règles.
DELETE /api/jobs/:id	Archiver ou supprimer selon la politique de conservation.

Statuts proposés pour une offre : draft, pending_review si modération activée, published, paused, closed, rejected. Définir explicitement les transitions autorisées et enregistrer l'auteur/date de chaque changement.

8. Gestion des candidatures et entretiens

Endpoint	Fonction
POST /api/jobs/:id/applications	Créer une candidature ; vérifier que l'offre est publiée et ouverte.
GET /api/candidate/applications	Lister les candidatures du candidat connecté.
GET /api/recruiter/applications	Lister les candidatures des offres des entreprises autorisées.
GET /api/applications/:id	Détail accessible au candidat propriétaire ou recruteur autorisé.
PATCH /api/applications/:id/status	Changer le statut si la transition est permise et tracer l'historique.
POST /api/applications/:id/interviews	Planifier un entretien.
PATCH /api/interviews/:id	Modifier ou annuler un entretien autorisé.

Workflow indicatif : submitted → reviewing → shortlisted → interview → accepted ou rejected ; withdrawn peut être proposé au candidat si la règle est validée. Chaque transition doit être contrôlée côté serveur et ajoutée à application_history.

9. CV, fichiers et favoris

- CV : upload, remplacement, téléchargement autorisé et suppression logique/physique contrôlée.
- Contrôler taille maximale, extension et type MIME réel ; ne pas se fier uniquement au nom du fichier.
- Renommer les fichiers avec un identifiant généré ; empêcher l'exécution de fichiers uploadés.
- Stocker les fichiers hors du dossier public du serveur et servir via endpoint contrôlé ou URL signée.
- Favoris : POST /api/jobs/:id/favorite, DELETE /api/jobs/:id/favorite, GET /api/candidate/favorites.
- Empêcher les candidatures multiples à une même offre si la règle métier le prévoit, via contrainte unique et transaction.

10. Notifications et communication

- Créer une notification lors d'événements métier : nouvelle candidature, changement de statut, invitation à un entretien, rappel si activé.
- Endpoints : GET /api/notifications, PATCH /api/notifications/:id/read, PATCH /api/notifications/read-all.
- Vérifier que l'utilisateur ne consulte et ne modifie que ses propres notifications.

- Email et notifications temps réel (Socket.io) sont des extensions optionnelles, à intégrer après stabilisation des workflows.

11. Contrat API et gestion des erreurs

Préfixe versionné recommandé : /api/v1. Utiliser des noms de ressources cohérents, des codes HTTP standards et une pagination stable.

```
Réponse succès :
{
"success": true,
"message": "Opération réussie",
"data": {}
}

Réponse erreur :
{
"success": false,
"message": "Données invalides",
"errors": []
}
```

Code HTTP	Usage
200	Lecture ou modification réussie.
201	Ressource créée.
204	Suppression/révocation sans corps de réponse.
400	Requête mal formée.
401	Authentification absente ou invalide.
403	Utilisateur authentifié mais non autorisé.
404	Ressource inexistante ou non accessible selon politique.
409	Conflit : email existant, candidature déjà créée, etc.
422	Échec de validation métier.
429	Trop de requêtes.
500	Erreur serveur inattendue.

Centraliser le middleware d'erreurs. En production, ne jamais renvoyer les traces de pile ou détails SQL au client.

12. Sécurité et protection des données

- Validation serveur de chaque entrée (body, params, query).
- Requêtes SQL paramétrées ou ORM ; jamais de concaténation non maîtrisée.
- Helmet, CORS limité aux origines autorisées, rate limiting et politique de cookies cohérente.
- JWT avec expiration courte ; refresh token protégé, rotation/révocation si implémentée.
- RBAC + contrôle de propriété sur chaque ressource privée.
- Protection contre IDOR, injection SQL, XSS stockée, brute force et upload malveillant.
- Secrets dans .env non versionné ; fournir .env.example sans valeurs sensibles.
- Logs sans mot de passe, token, CV ou données personnelles inutiles.
- Sauvegardes, politique de rétention et suppression des données à définir.

13. Tests, documentation et observabilité

- Tests unitaires des services et validateurs.
- Tests d'intégration des endpoints avec une base de test isolée.
- Cas positifs et négatifs : utilisateur anonyme, rôle incorrect, propriétaire différent, ressource inexistante, champs invalides, doublons.
- Swagger/OpenAPI décrivant routes, paramètres, schémas, codes d'erreur et authentification.
- Logs structurés avec niveau (info/warn/error), identifiant de requête et masquage des données sensibles.
- Scripts npm : dev, start, test, migrate, seed et lint si configuré.

14. Déploiement et configuration

- Environnements séparés : development, test et production.
- Variables : PORT, DATABASE_URL ou DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASSWORD, JWT_SECRET, FRONTEND_URL, STORAGE_* et MAIL_* selon services activés.
- Connexion PostgreSQL avec pool, timeout et gestion propre des erreurs.
- Migrations exécutées de manière contrôlée avant déploiement ; seeds uniquement pour données initiales de test.
- HTTPS en production, gestion des logs, sauvegardes et stratégie de restauration.
- README backend : prérequis, installation, configuration .env, migrations, lancement, tests, documentation API.

15. Critères d'acceptation et ordre de réalisation

Critères d'acceptation

- Les rôles ne peuvent exécuter que les actions autorisées.
- Les données privées ne sont jamais accessibles par simple modification d'un ID.
- Les workflows d'offre et de candidature respectent les transitions définies.
- Toutes les entrées sont validées et les erreurs ont un format cohérent.
- Les migrations fonctionnent sur une base vide et les tests critiques passent.
- La documentation Swagger correspond au comportement réel de l'API.

Ordre conseillé

- 1. Initialisation Express, configuration et connexion PostgreSQL.
- 2. Migrations et modèle relationnel.
- 3. Authentification, profils et RBAC.
- 4. Entreprises et membres.
- 5. CRUD des offres et recherche.
- 6. Candidatures et historique de statuts.
- 7. CV/fichiers et favoris.
- 8. Entretiens et notifications.
- 9. Administration, sécurité renforcée, tests et documentation.
- 10. Déploiement et sauvegardes.